

MIRAGE: Mobile Agents with Implicit Reasoning and Generative World Models

Zhichao Yang¹ Yuanze Hu¹ Haojie Hao¹ Longkun Hao²
 Dongshuo Huang² Hongyu Lin³ Gen Li¹ Lanqing Hong⁴
 Yihang Lou^{5,†} Yan Bai^{6,†}

¹Beihang University

²Northwestern Polytechnical University

³Institute of Software, Chinese Academy of Sciences

⁴National University of Singapore

⁵Peking University

⁶Independent Researcher

[†]Corresponding authors.

Abstract

Mobile agents are increasingly expected to operate everyday applications from screenshots and language goals, where reliable control requires reasoning over screen affordances, multi-step navigation, and future state changes. Yet many agents externalize this computation as long textual thoughts, making interaction slower, supervision more costly, and deployment less efficient. We introduce MIRAGE (Mobile agents with Implicit Reasoning And Generative world modELs), a framework that learns continuous latent reasoning representations from visible textual thoughts. MIRAGE introduces an efficient latent-space learning procedure that transfers explicit reasoning into compact hidden states, allowing the agent to reason internally without decoding long rationales. It further brings a world-model perspective into mobile-agent training: the model’s latent reasoning vectors are aligned with future screenshots, encouraging the agent to predict upcoming interface states in latent space before executing an action. This makes the hidden computation not only a compressed thought trace, but also a forward-looking representation of how the environment may change. At inference time, MIRAGE reasons in continuous latent space, reducing token generation while improving execution efficiency. On AndroidWorld, MIRAGE matches explicit-CoT SFT in the 4B ablation under a $3\text{--}5\times$ lower decoded-token budget, and improves a comparable instruction-tuned baseline by 10.2 points; on AndroidControl, it improves action grounding with over 75% fewer generated tokens.

1 Introduction

As vision-language models improve, more mobile-agent systems now use them to execute mobile operations directly from screenshots and user instructions. Recent systems such as UI-TARS, MAI-UI, OS-ATLAS, and SeeClick train VLMs to understand GUI screens and produce taps, swipes, text entry, or navigation commands [1–4]. Yet turning a screen-level VLM into a reliable mobile agent still hinges on navigation reasoning: the model must track task progress, decide which screen to visit next, and anticipate how each action will change the interface. Figure 1 highlights this reasoning bottleneck. Current mobile agents often make this reasoning explicit through long thoughts or verbose action traces, which increases decoding time, context usage, and supervision cost. Their execution accuracy remains limited, suggesting that mobile agents need reasoning that preserves explicit-trace

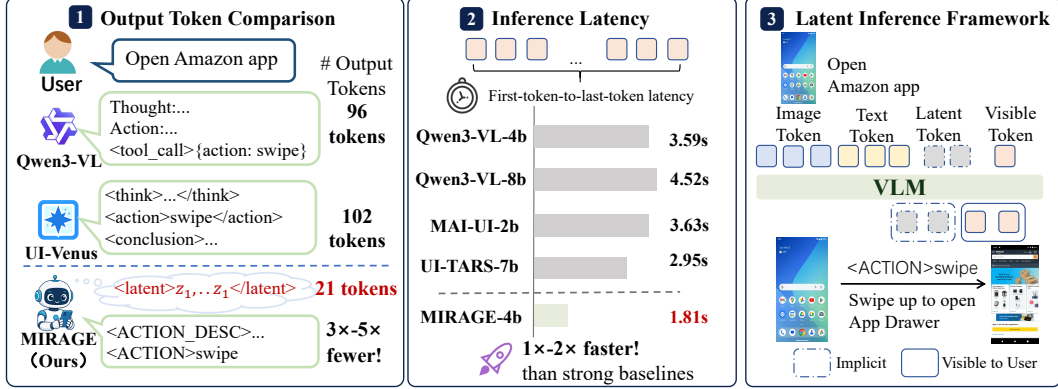


Figure 1: Inference-time comparison on a randomly sampled task and MIRAGE workflow: baselines emit verbose visible traces, while MIRAGE reasons with latent tokens and exposes concise actions.

capability while being cheaper to run under realistic deployment constraints, where every extra token slows the interaction and delays feedback during interactive control.

To address these issues, we propose MIRAGE, whose central idea is that a mobile agent should learn to “think forward” inside the model. Before emitting an action, the agent forms a compact internal representation of what the current screen affords, why an action is appropriate, and how the interface is likely to change afterwards. This reasoning is carried out in latent space rather than decoded as visible text, so MIRAGE avoids emitting intermediate thoughts and reduces both output tokens and first-token-to-last-token latency. MIRAGE uses a two-stage training procedure. In the first stage, it trains on explicit text traces so the backbone learns the mobile action space and how to express observation, rationale, and future-screen prediction. In the second stage, MIRAGE replaces the textual reasoning block with continuous latent reasoning slots and continues training, gradually teaching the model to move reasoning into latent space. At inference time, only action tokens are decoded; no rationale text is emitted and the interaction latency is substantially reduced.

To transfer explicit reasoning capability into latent space, MIRAGE introduces Approximate Parallel Latent Refinement (APLR), which refines all latent slots in parallel through K Jacobi-style rounds, approximating full sequential latent chain-of-thought: the first K slots provably match the serial rollout while the remaining tail error is bounded. To further enable the model to internalize future-state prediction, MIRAGE attaches a lightweight Q-Former world-model head that aligns the output latent representations with stop-gradient visual features of the next screenshot from the frozen vision encoder. This alignment teaches the agent to anticipate GUI state transitions, prevents latent representation collapse, and supplies dense supervision that compensates for APLR’s bounded tail error. Crucially, the action cross-entropy loss and the next-frame feature-alignment loss are optimized jointly, so the same latent states become both action-discriminative and transition-predictive. This coupling lets MIRAGE retain CoT-level reasoning capacity while moving the intermediate computation out of the decoded text stream, yielding comparable explicit-reasoning capability at substantially lower inference cost.

MIRAGE yields strong results on established mobile-agent benchmarks. On AndroidWorld, MIRAGE improves task success by 10.2 percentage points over a comparable instruction-tuned baseline and matches explicit CoT in the 4B ablation under a 3–5 $\times$ lower decoded-token cost. On AndroidControl, it improves action-grounding accuracy while reducing generated tokens by over 75%, demonstrating that latent reasoning and future-state alignment preserve explicit-reasoning capability with substantially leaner inference.

This paper makes three contributions:

- We present MIRAGE, a mobile agent that reasons entirely in latent space, producing far fewer output tokens and substantially lower inference latency while matching explicit-CoT task performance in the 4B ablation under a 3–5 $\times$ lower decoded-token cost.

- We introduce APLR, a parallel Jacobi-style latent refinement procedure that approximates full serial latent chain-of-thought at a fraction of the training cost, with a provable bound on the approximation error of the tail slots.
- We introduce a Q-Former world-model head that aligns latent reasoning states with future-screenshot features in latent space, enabling the agent to predict upcoming GUI transitions and directly improving task capability.

2 Related Work

Mobile and GUI agents. GUI-agent benchmarks have evolved from web interaction to dynamic Android control, covering grounded shopping, open-web navigation, data-scaling studies, and online task completion [5–8]. Recent systems specialize VLMs for screenshot grounding, GUI actions, and mobile-device operation [1, 3, 4, 9]. Instead of extending grounding or planning pipelines, MIRAGE trains internal agent states by replacing visible reasoning traces with latent slots predictive of the next GUI state during action decoding and transition modeling.

Reasoning in language and vision-language agents. Visible CoT and ReAct-style traces improve reasoning and acting but expose long rationales and consume context [10, 11]. Other work internalizes computation through pause tokens, private thoughts, distillation, or continuous latent CoT [12–15]. MIRAGE brings implicit reasoning to mobile GUI control, where latent thoughts support action selection and transition understanding, using APLR to approximate serial latent refinement in a strictly causal triangular system rather than an equilibrium model.

World models and visual feature prediction. World models learn predictive dynamics for control, from compact latent simulators to latent-imagination agents [16, 17]. Joint-embedding vision objectives show that feature prediction can learn semantics without pixel generation [18, 19], while modal-alignment work highlights bottlenecks between visual and language representations [20]. Q-Former queries offer a lightweight cross-attention bottleneck [21]. Recent GUI world models predict future screens, sketches, or semantic states [22, 23]. MIRAGE instead uses future prediction to shape latent reasoning states, encouraging action-induced transition representations without generating pixels or future text at inference time.

3 Method

3.1 Problem Formulation: From Explicit-Thought to Latent-Thought Mobile Agents

At interaction step t , a mobile GUI agent observes the current screenshot x_t , a user instruction u , and an interaction history $h_t = (a_{<t}, x_{<t})$. It outputs an action a_t , such as tapping a coordinate, typing text, scrolling, or navigating back. We serialize actions as text tokens so that a VLM can model mobile control as conditional generation:

$$o_t = (x_t, u, h_t), \quad p_\theta(a_t \mid o_t). \quad (1)$$

Mobile GUI agents face a critical trade-off between reasoning capability and deployment efficiency. Explicit chain-of-thought improves multi-step decision making because the model can verbalize observations, rationales, and predicted state transitions before acting; however, this reasoning is paid for with visible tokens. Many explicit-thought mobile agents first generate a readable reasoning trace τ_t and then generate the action. This factorization can be written as

$$p_\theta(\tau_t, a_t \mid o_t) = \prod_{i=1}^{|\tau_t|} p_\theta(\tau_t^{(i)} \mid \tau_t^{(<i)}, o_t) \cdot \prod_{j=1}^{|a_t|} p_\theta(a_t^{(j)} \mid \tau_t, a_t^{(<j)}, o_t). \quad (2)$$

The intermediate thought τ_t is useful, but it is also a sequence of discrete text tokens: it consumes context budget, increases decoding latency, and usually requires human-written or synthetic rationale supervision. This makes explicit CoT attractive as a training scaffold but costly as an inference-time interface for mobile deployment.

To resolve this tension, MIRAGE formulates mobile control as a latent-thought generation problem. Instead of removing reasoning, we internalize it: the model keeps a structured computation budget, but

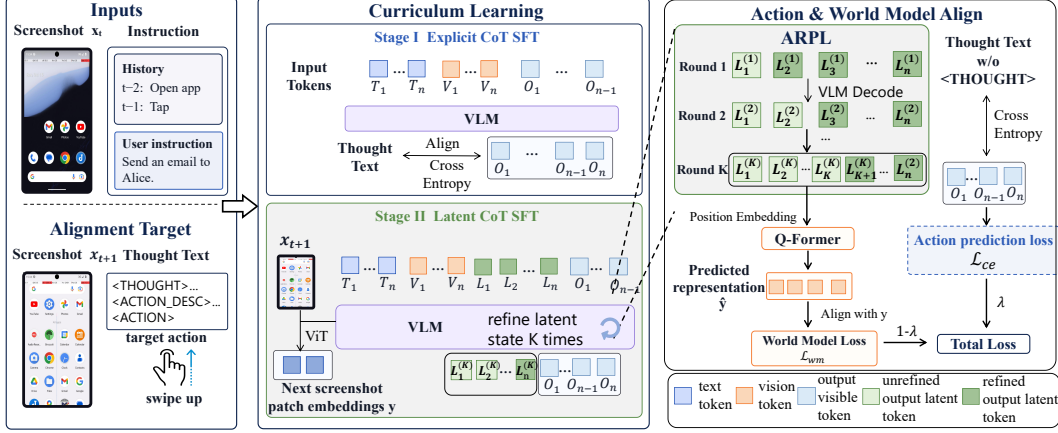


Figure 2: MIRAGE pipeline. Stage 1 learns explicit mobile thoughts and action formatting. Stage 2 replaces the thought text with latent slots, refines them with APLR, and trains a Q-Former world model to align latent states with next-frame visual features.

the intermediate computation is carried by hidden states rather than text. We introduce N continuous latent variables $\mathbf{z}_t = (z_{t,1}, \dots, z_{t,N}), z_{t,i} \in \mathbb{R}^d$, in the decoder context:

$$p_\theta(a_t | o_t) = \int p_\theta(a_t | \mathbf{z}_t, o_t) q_\theta(\mathbf{z}_t | o_t) d\mathbf{z}_t. \quad (3)$$

Unlike explicit CoT, $\mathbf{z}_t$ does not live in the vocabulary and is never detokenized. Its inference distribution q_θ is computed implicitly by decoder hidden states, so latent-thought training does not need rationale-text supervision. This formulation gives four practical advantages: it preserves context by removing explicit rationale tokens, speeds up inference by shortening the decoded sequence, retains multi-step reasoning capacity through latent slots, and exposes the hidden reasoning state to auxiliary regularization such as next-frame prediction. In this sense, latent thoughts bridge the gap between CoT-style reasoning strength and efficient mobile-agent deployment, motivating the Approximate Parallel Latent Refinement (APLR) refinement and world-model regularization developed next.

3.2 Training Latent Chain-of-Thought via Approximate Parallel Latent Refinement

3.2.1 Structured Thought Data and Latent Insertion

Our training data uses a structured thought format with three mobile-agent reasoning dimensions:

```
<THOUGHT> [observation] [rationale] [predict] <ACTION_DESC> ...
<ACTION> ...
```

The observation field describes the visible screen state, rationale explains why a particular operation should be taken, and predict describes the expected next-screen transition. Explicit-thought warmup uses this structured text as ordinary next-token supervision. Latent-thought training then replaces the entire <THOUGHT> block with N latent tokens. The token sequence becomes

$$[\text{ctx}] ; [\text{start}] ; \underbrace{\langle \text{lat} \rangle, \dots, \langle \text{lat} \rangle}_{N \text{ slots}} ; [\text{end}] ; \langle \text{ACTION_DESC} \rangle \dots \langle \text{ACTION} \rangle. \quad (4)$$

The latent slots occupy positions $p_1 < \dots < p_N$ and share the learned initializer e_{lat} .

3.2.2 From Serial Masking to Approximate Parallel Latent Refinement

Original Coconut-style latent reasoning [15] replaces visible reasoning steps with continuous hidden states and computes those states serially; Appendix D gives the full formalization. Our APLR keeps the same latent-thinking motivation but changes the execution pattern: rather than advancing through thought steps one by one, it treats the thought block as N slots and refines all slots in parallel rounds.

With APLR, latent reasoning becomes a small number of synchronous refinement passes instead of a long serial rollout. Conceptually, this lets the model keep an internal multi-step computation budget while making training cost depend on the refinement depth K , not directly on the latent length N .

With fixed non-latent context $c = o_t$ and latent positions $p_1 < \dots < p_N$, the serial target states are

$$s_i = G_i(s_1, \dots, s_{i-1}; c), \quad i = 1, \dots, N, \quad (5)$$

where G_i is the hidden state at position $p_i - 1$ when the previous latent states have already been written into the sequence. This is a forward-substitution procedure over a causal triangular system. It is accurate, but expensive: fully refining all N latent tokens requires one full decoder pass per latent update, plus a final pass for action logits.

We use APLR to eliminate the need for N serial forward passes in traditional latent CoT training. The key observation is that causal attention makes the latent system strictly triangular: z_i cannot depend on itself or on any future $z_{j>i}$. Therefore, instead of filling the slots one by one, APLR performs Jacobi-style rounds:

$$z_i^{(k+1)} = G_i(z_1^{(k)}, z_2^{(k)}, \dots, z_{i-1}^{(k)}; c), \quad i = 1, \dots, N, \quad k = 0, \dots, K-1. \quad (6)$$

All right-hand sides use old round- k values, so one full forward updates all slots. In practice, we use a small refinement budget, $K = 3$ by default.

APLR has a precise relationship to the serial refinement in (5). After K parallel rounds, the first K latent slots exactly match the serial solution, $z_i^{(K)} = s_i$ for $i \leq K$. Compared with refining all latent tokens one by one, the unrefined tail slots retain a structured residual

$$\delta^{(K)} \approx A^K \delta^{(0)}, \quad (7)$$

where A is a strictly lower-triangular Jacobian. Appendix F proves that after K APLR rounds, the first K latent slots exactly recover the serial solution, and then derives the tail-error form in (7). In implementation, early bootstrap passes may run without gradient, while the first pass can retain gradient through e_{lat} for stability. Before the final gradient-enabled pass, we rebuild the input embeddings with a mask: latent positions use detached bootstrap values, while non-latent positions retain their original gradient lineage to the vision tower and token embeddings.

The final APLR pass is trained with standard next-token CE over serialized action tokens:

$$\mathcal{L}_{\text{ce}} = - \sum_{j=1}^{|a_t|} \log p_\theta \left(a_t^{(j)} \mid \mathbf{z}_t^{(K)}, a_t^{(<j)}, o_t \right). \quad (8)$$

When $K \ll N$, the tail latents $z_{K+1:N}^{(K)}$ retain approximation error relative to a trainer that refines every latent token serially. In our structured thought format, the `predict` dimension concerns how the screen will change after the action and typically lies late in the thought sequence. Action CE only penalizes tail-error directions that affect the next action tokens; it does not directly supervise errors that matter for environment prediction. This motivates the world-model objective.

3.3 World Model: Q-Former Next-Frame Alignment

The `predict` field describes future screen semantics after a mobile action. Unlike GUI world models that generate pixels or sketches [22], our Q-Former head directly regularizes latent reasoning states to predict future GUI features, forcing latent slots to encode environment dynamics rather than only current-screen action correlations. Tail-error directions that harm future-screen prediction therefore receive direct gradients; Appendix G formalizes this with a local second-order argument.

Because the training records are sequential, most non-terminal step- t examples provide the next screenshot x_{t+1} . We use it as feature supervision, not as a pixel-generation target: x_{t+1} is passed through the VLM’s own frozen vision tower, yielding a stable next-frame feature target that does not drift with the world-model head.

We gather the final-pass hidden states at latent positions to obtain latent CoT hidden states

$$C_t = \text{Gather} \left(H_t^{(K)}, p_1, \dots, p_N \right) \in \mathbb{R}^{L \times H}, \quad (9)$$

where each row is one latent thought vector after APLR. The next screenshot is first converted by the VLM image processor into visual patches. After the VLM’s spatial merge operation, each target patch has both a feature vector and a 2D grid coordinate. We write the target features as

$$V_{t+1} = \text{sg}(f_{\text{vis}}(x_{t+1})) = [v_1, \dots, v_M] \in \mathbb{R}^{M \times H_v}, \quad (10)$$

where $\text{sg}(\cdot)$ denotes stop-gradient. The stop-gradient prevents collapse through the target branch, no image decoder is required, and the target lives on the same visual manifold as the backbone features.

For each next-frame patch j with grid coordinate (r_j, c_j) , we build a learnable spatial query

$$q_j = e_{r_j}^{\text{row}} + e_{c_j}^{\text{col}} \in \mathbb{R}^{d_q}. \quad (11)$$

This says, in simple terms, “predict the feature at row r_j and column c_j of the next screen.” The separable row/column design gives a stable 2D positional prior even when mobile screenshots have different resolutions and therefore different patch grids.

The BLIP-2-style Q-Former aligner [21] self-attends over queries, cross-attends to C_t , and projects each query output into the VLM feature space:

$$U_t = \text{QFormer}_{\phi}(Q_t, C_t), \quad \hat{V}_{t+1} = W_{\text{vis}} U_t. \quad (12)$$

The output $\hat{V}_{t+1} = [\hat{v}_1, \dots, \hat{v}_M]$ is therefore a predicted next-frame representation, one vector per next-frame patch. The default objective is masked per-patch cosine distance:

$$\mathcal{L}_{\text{wm}} = \frac{1}{|\mathcal{M}|} \sum_{(b,j) \in \mathcal{M}} \left(1 - \frac{\langle \hat{v}_{b,j}, v_{b,j} \rangle}{\|\hat{v}_{b,j}\|_2 \|v_{b,j}\|_2} \right). \quad (13)$$

The mask $\mathcal{M}$ keeps only valid patches from non-terminal latent samples; cosine is default, with MSE used for ablations.

3.4 Two-Stage Training Pipeline

We use a curriculum learning strategy to make latent reasoning learnable rather than asking the model to discover a continuous thought space from scratch. Stage 1 warm-ups the model on explicit structured thoughts to learn action formatting and observation–rationale–prediction reasoning patterns. Specifically, the full <THOUGHT> trace is exposed as text supervision and the VLM is trained with standard next-token CE over the structured thought, action description, and action tokens. Stage 2 distills this explicit reasoning process into compact latent slots: the <THOUGHT> block is replaced by the latent sequence in (4), APLR refines the latent states ($K = 3$ by default), the collator loads next screenshots, and the world model regularizes the latent slots through Q-Former next-frame feature alignment. The joint objective is

$$\mathcal{L} = \lambda \mathcal{L}_{\text{ce}} + (1 - \lambda) \mathcal{L}_{\text{wm}}, \quad \lambda \in (0, 1). \quad (14)$$

We use $\lambda = 0.8$ by default. Figure 2 summarizes the pipeline, and Appendix E gives pseudocode. At inference time, the agent only performs latent substitution and greedy action decoding; the Q-Former head is used only for training-time representation shaping [24].

4 Experiments

4.1 Setup

Backbones. We fine-tune two vision-language models from the Qwen3-VL family [25]: **Qwen3-VL-4B-Instruct** (4B parameters) and **Qwen3-VL-8B-Instruct** (8B parameters). Both backbones share the same action-serialization vocabulary, latent-slot token format, and Q-Former world-model head. Their latent computation budgets differ: the main Qwen3-VL-4B setting uses 9 latent slots with 3 APLR refinement passes, while the Qwen3-VL-8B setting uses 6 latent slots with 3 refinement passes.

Evaluation benchmarks. We evaluate on two standard mobile-agent benchmarks. **AndroidControl** [7] provides paired high-level and low-level instructions with ground-truth action sequences, allowing separate measurement of instruction-following exact match (EM) and action accuracy. **AndroidWorld** [8] is a dynamic, on-device benchmark spanning 116 real-world task instances across 20 Android apps, measuring end-to-end task completion rate under live Android dynamics.

Table 1: AndroidControl results. EM = exact match, Action Acc. = action accuracy, and Tokens = average generated tokens per step. External rows use reported Type/EM metrics; green parentheses show MIRAGE gains over size-matched Qwen3-VL-Instruct baselines. Best primary metrics and lowest Tokens are in **bold**.

Model	Size	Low-Level			High-Level		
		EM	Action Acc.	Tokens	EM	Action Acc.	Tokens
GPT-4o	–	66.3	20.8	–	74.3	19.4	–
GUI-R1-3B	3B	58.0	46.6	–	83.7	64.4	–
UI-R1-3B	3B	57.9	45.4	–	72.5	57.4	–
GUI-R1-7B	7B	71.6	51.7	–	85.2	66.5	–
ShowUI-2B	2B	74.83	81.68	31.57	59.82	74.18	34.39
MAI-UI-2B	2B	74.17	64.70	32.87	67.3	64.30	32.78
UI-Venus-Navi-7B	7B	85.09	93.05	103.66	75.2	86.14	96.36
UI-TARS-7B-SFT	7B	76.28	82.16	91.80	62.12	74.37	91.38
Qwen3-VL-4B-Instruct	4B	68.48	75.15	115.67	54.58	69.39	116.00
Qwen3-VL-8B-Instruct	8B	77.66	82.54	79.86	60.78	74.66	80.52
MIRAGE-4B	4B	77.59 (↑13.30%)	91.09 (↑21.21%)	18.92 (↓83.64%)	64.43 (↑18.05%)	75.67 (↑9.05%)	20.61 (↓82.23%)
MIRAGE-8B	8B	83.75 (↑7.84%)	94.62 (↑14.64%)	18.01 (↓77.45%)	72.35 (↑19.04%)	82.83 (↑10.94%)	19.89 (↓75.30%)

Table 2: AndroidWorld results. SR = task success rate; Avg. Steps/Tokens are per task. Green parentheses show MIRAGE gains over size-matched Qwen3-VL-Instruct baselines; red marks higher step count. Best SR and lowest averages are in **bold**.

Model	Size	SR	Avg. Steps	Avg. Tokens
GPT-4o	–	30.6	–	–
Ferret-UI Lite	3B	28.0	–	–
UI-TARS	7B	33.0	–	–
ShowUI-2B	2B	7.8	18.9	33.0
MAI-UI-2B	2B	46.5	15.0	116.0
UI-Venus-Navi-7B	7B	42.2	13.3	92.0
UI-TARS-7B-SFT	7B	33.6	17.3	104.0
Qwen3-VL-4B-Instruct	4B	42.9	14.3	103.0
Qwen3-VL-8B-Instruct	8B	47.6	12.6	108.0
MIRAGE-4B	4B	52.6 (↑22.6%)	14.2 (↓0.7%)	31.0 (↓69.9%)
MIRAGE-8B	8B	57.8 (↑21.3%)	13.7 (↑8.7%)	27.0 (↓75.0%)

Baselines. We compare against size-matched **Qwen3-VL-4B/8B-Instruct** backbones [25], a general multimodal baseline **GPT-4o** [26], reinforcement-tuned GUI agents **GUI-R1/UI-R1** [27, 28], and recent GUI-agent systems including **ShowUI**, **MAI-UI**, **UI-Venus-Navi**, **UI-TARS-7B-SFT**, and **Ferret-UI Lite** across AndroidControl and AndroidWorld [1, 2, 29–31].

4.2 Main Results

AndroidControl. Table 1 shows that MIRAGE improves the size-matched Qwen3-VL-Instruct baselines while producing much shorter action outputs. On the low-level split, MIRAGE-4B improves EM from 68.48 to 77.59 and action accuracy from 75.15 to 91.09, while reducing the average generation length from 115.67 to 18.92 tokens. MIRAGE-8B shows the same pattern, increasing low-level EM from 77.66 to 83.75 and action accuracy from 82.54 to 94.62 with 18.01 tokens per step. On the high-level split, MIRAGE-4B improves EM/action accuracy by 9.85/6.28 percentage points over Qwen3-VL-4B-Instruct, and MIRAGE-8B improves them by 11.57/8.17 points over Qwen3-VL-8B-Instruct. These gains suggest that replacing verbose visible thoughts with latent reasoning can improve grounding and action selection without increasing the decoded token budget.

AndroidWorld. Table 2 evaluates the same models in a dynamic on-device setting. MIRAGE raises AndroidWorld SR from 42.9 to 52.6 for 4B and from 47.6 to 57.8 for 8B, while reducing average tokens from 103.0/108.0 to 31.0/27.0. The 8B model uses slightly more steps than its Qwen3-VL baseline, so the gain does not simply come from shorter trajectories. Among retained specialized GUI

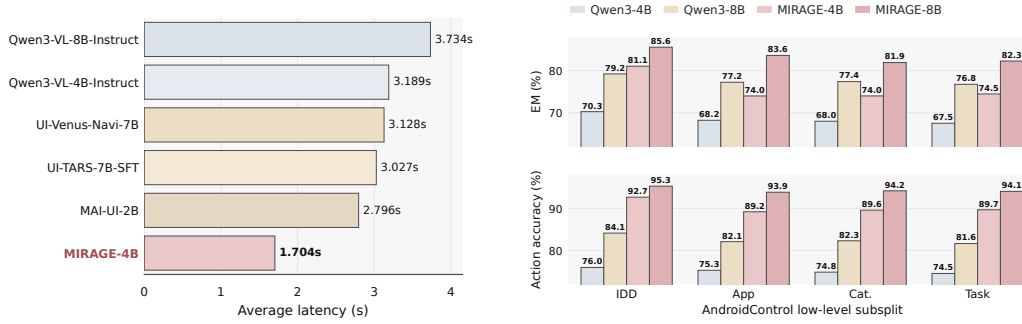


Figure 3: **Left:** Average latency from the first generated token to the final generated token. MIRAGE-4B produces the shortest decoded sequence latency among the compared models. **Right:** AndroidControl low-level subsplit EM and action accuracy, corrected by subtracting each model’s offset between the raw low-level subsplit average and the reported low-level all-split score.

agents, MIRAGE-8B gives the highest AndroidWorld SR with far fewer generated tokens. Together, the two benchmarks show that latent reasoning and world-model training improve task effectiveness while keeping outputs compact.

Figure 3 further analyzes efficiency and robustness. The left panel measures first-to-last-token latency; MIRAGE-4B is fastest, consistent with the token reductions in Tables 1 and 2. The right panel breaks AndroidControl low-level results into IDD, app-unseen, category-unseen, and task-unseen subsplits. After correcting each model to its reported all-split score, MIRAGE remains strong across subsplits, especially on action accuracy, suggesting that Table 1 is not driven by a single easy split.

4.3 Ablation Study

We ablate the three core components of MIRAGE—latent CoT slots, APLR parallel refinement, and the Q-Former world-model head—using Qwen3-VL-4B-Instruct on AndroidWorld.

Ablation analysis. Figure 4 reports cross-entropy loss for the same Qwen3-VL-4B training variants summarized in Table 3. For the latent settings, we allocate 9 latent slots; in the serial latent-CoT reference, the observation, rationale, and prediction fields each correspond to three latent slots and are masked through a stepwise curriculum. The sharp change after the first epoch is expected rather than a failure mode: training switches from explicit-thought warmup to latent-CoT training at that point, and the learning rate is reset with the new stage. After this transition, APLR without the world-model objective exhibits a higher loss than the serial latent-CoT reference, consistent with the fact that parallel approximation leaves tail latent states less directly supervised by action CE alone. Adding the Q-Former world-model objective makes the APLR curve track the serial latent-CoT trend more closely, suggesting that next-frame feature alignment supplies useful gradients for the transition-predictive parts of the latent state.

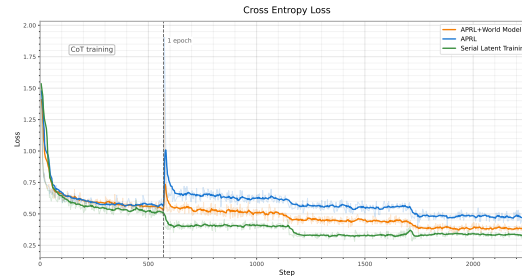


Figure 4: Cross-entropy training loss for Qwen3-VL-4B variants matched to Table 3.

Table 3 shows the same pattern in final AndroidWorld SR. Action-only SFT performs worse than the base model, indicating that removing thought supervision without replacing it by internal computation can harm interactive decision making. Explicit CoT SFT and full MIRAGE-4B both reach 52.6 SR, showing that latent reasoning can match explicit CoT while avoiding

Table 3: Component ablation on AndroidWorld (Qwen3-VL-4B, SR %).

Variant	Lat.	APLR	WM	SR
Base model	×	×	×	42.9
Action-only SFT	×	×	×	31.0
Explicit CoT SFT	×	×	×	52.6
Latent CoT, serial	✓	×	×	50.9
8APLR only	✓	✓	×	48.2
MIRAGE-4B	✓	✓	✓	52.6

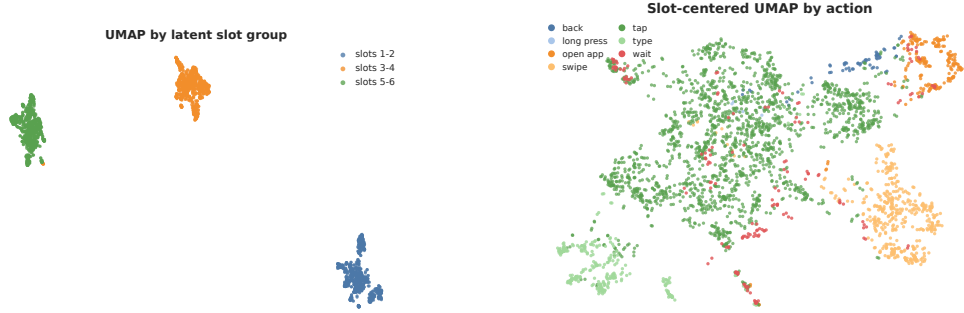


Figure 5: **Left:** UMAP by latent slot group. **Right:** slot-centered UMAP by action type after subtracting per-slot means.

decoded rationale tokens. Serial latent CoT preserves much of this benefit (50.9 SR), and APLR without the world-model objective reaches 48.2; adding the Q-Former world-model objective restores the explicit-CoT-level result while keeping reasoning latent at inference time.

We study the sensitivity of MIRAGE to the number of latent slots L , APLR refinement passes K , and the loss balance λ_{ce} on AndroidWorld. Table 4 shows that the latent computation budget matters.

Table 4: AndroidWorld ablations over latent-slot budget, APLR refinement passes, and loss balance. SR is task success rate (%). Unless otherwise noted, $\lambda_{ce} = 0.8$.

Model	Latent slots	Ref. passes	λ_{ce}	SR (%)
MIRAGE-8B	6	2	0.8	46.6
MIRAGE-8B	6	3	0.8	57.8
MIRAGE-4B	9	3	0.8	52.6
MIRAGE-4B	3	3	0.8	32.8
MIRAGE-4B	9	3	0.1	48.3

For MIRAGE-8B, increasing APLR refinement from two to three passes improves AndroidWorld SR from 46.6 to 57.8, suggesting that the third pass substantially reduces harmful tail-latent error. For MIRAGE-4B, reducing the latent budget from 9 to 3 slots lowers SR from 52.6 to 32.8, showing that the mobile-agent thought state needs enough continuous capacity to represent observation, rationale, and future-screen prediction. Finally, setting $\lambda_{ce} = 0.1$ decreases SR from 52.6 to 48.3, consistent with the intended role of the world model as an auxiliary regularizer rather than the dominant training objective.

4.4 Latent Reasoning Visualization

We analyze a MIRAGE-8B checkpoint with six latent slots on the AndroidControl IDD split; Appendix C provides full t-SNE and per-slot action projections. Figure 5 shows that latent training does not collapse to a single undifferentiated representation. Before slot-centering, slots 1–2, 3–4, and 5–6 occupy three separated regions, corresponding to the observation, rationale, and prediction dimensions of the structured thought. After subtracting the slot mean, examples remain organized by action type: open-app, swipe, type, and tap states form distinguishable regions in the centered latent space. These visualizations indicate that MIRAGE learns both position-specific reasoning subspaces and action-discriminative latent states.

5 Limitations, Broader Impact, and Conclusion

MIRAGE replaces visible rationales with APLR-refined latent slots regularized by next-frame features, preserving explicit-CoT capability at much lower decoding cost. Limitations include supervised-only training, feature-level world modeling, next-frame supervision, and the need for privacy and action safeguards before deployment.

References

- [1] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.
- [2] Hanzhang Zhou, Xu Zhang, Panrong Tong, Jianan Zhang, Liangyu Chen, Quyu Kong, Chenglin Cai, Chen Liu, Yue Wang, Jingren Zhou, et al. Mai-ui technical report: Real-world centric foundation gui agents. *arXiv preprint arXiv:2512.22047*, 2025.
- [3] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, et al. Os-atlas: A foundation action model for generalist gui agents. *arXiv preprint arXiv:2410.23218*, 2024.
- [4] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing gui grounding for advanced visual gui agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9313–9332, 2024.
- [5] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757, 2022.
- [6] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36:28091–28114, 2023.
- [7] Wei Li, William Bishop, Alice Li, Chris Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on ui control agents. *Advances in Neural Information Processing Systems*, 37:92130–92154, 2024.
- [8] Christopher Rawles, Sarah Clinckemaulle, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, et al. Androidworld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573*, 2024.
- [9] Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. Mobile-agent: Autonomous multi-modal mobile device agent with visual perception. *arXiv preprint arXiv:2401.16158*, 2024.
- [10] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [11] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- [12] Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training language models with pause tokens. *arXiv preprint arXiv:2310.02226*, 2023.
- [13] Eric Zelikman, Georges Harik, Yijia Shao, Varuna Jayasiri, Nick Haber, and Noah D Goodman. Quiet-star: Language models can teach themselves to think before speaking. *arXiv preprint arXiv:2403.09629*, 2024.
- [14] Yuntian Deng, Kiran Prasad, Roland Fernandez, Paul Smolensky, Vishrav Chaudhary, and Stuart Shieber. Implicit chain of thought reasoning via knowledge distillation. *arXiv preprint arXiv:2311.01460*, 2023.
- [15] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024.

- [16] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2(3):440, 2018.
- [17] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019.
- [18] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 15619–15629, 2023.
- [19] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *arXiv preprint arXiv:2404.08471*, 2024.
- [20] Yuanze Hu, Zhaoxin Fan, Xinyu Wang, Gen Li, Ye Qiu, Zhichao Yang, Wenjun Wu, Kejian Wu, Yifan Sun, Xiaotie Deng, et al. Tinyalign: Boosting lightweight vision-language models by mitigating modal alignment bottlenecks. *arXiv preprint arXiv:2505.12884*, 2025.
- [21] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International conference on machine learning*, pages 19730–19742. PMLR, 2023.
- [22] Dezhao Luo, Bohan Tang, Kang Li, Georgios Papoudakis, Jifei Song, Shaogang Gong, Jianye Hao, Jun Wang, and Kun Shao. Vimo: A generative visual gui world model for app agents. *arXiv preprint arXiv:2504.13936*, 2025.
- [23] Yilin Cao, Yufeng Zhong, Zhixiong Zeng, Liming Zheng, Jing Huang, Haibo Qiu, Peng Shi, Wenji Mao, and Wan Guanglu. Mobiledreamer: Generative sketch world model for gui agent. *arXiv preprint arXiv:2601.04035*, 2026.
- [24] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-wam: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026.
- [25] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- [26] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.
- [27] Run Luo, Lu Wang, Wanwei He, Longze Chen, Jiaming Li, and Xiaobo Xia. Gui-r1: A generalist r1-style vision-language action model for gui agents. *arXiv preprint arXiv:2504.10458*, 2025.
- [28] Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Pengxiang Zhao, Guangyi Liu, et al. Ui-r1: Enhancing efficient action prediction of gui agents by reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 17608–17616, 2026.
- [29] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. Showui: One vision-language-action model for gui visual agent. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 19498–19508, 2025.
- [30] Zhangxuan Gu, Zhengwen Zeng, Zhenyu Xu, Xingran Zhou, Shuheng Shen, Yunfei Liu, Beitong Zhou, Changhua Meng, Tianyu Xia, Weizhi Chen, et al. Ui-venus technical report: Building high-performance ui agents with rft. *arXiv preprint arXiv:2508.10833*, 2025.
- [31] Zhen Yang, Zi-Yi Dou, Di Feng, Forrest Huang, Anh Nguyen, Keen You, Omar Attia, Yuhao Yang, Michael Feng, Haotian Zhang, et al. Ferret-ui lite: Lessons from building small on-device gui agents. *arXiv preprint arXiv:2509.26539*, 2025.

- [32] Yuxiang Chai, Siyuan Huang, Yazhe Niu, Han Xiao, Liang Liu, Guozhi Wang, Dingyu Zhang, Shuai Ren, and Hongsheng Li. Amex: Android multi-annotation expo dataset for mobile gui agents. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 2138–2156, 2025.

A Additional Method Details

Q-Former target construction. The world-model target is built online during training. For each sample with a valid next frame, the collator loads x_{t+1} and the base VLM vision stack extracts post-spatial-merge patch features. The Q-Former aligner never receives these target features as input; it receives only latent CoT hidden states and row/column query embeddings.

No target leakage. The world-model predictor receives only the final latent hidden states from the current observation and history. Next-frame pixels are used only on the detached target side of the feature-alignment loss, and missing next frames are masked out.

B Training Data and Schedule

Training data. We train on two sources of mobile-interaction data. First, we sample a subset of AMEX [32], the Android Multi-annotation Expo dataset, which provides 104K high-resolution screenshots annotated with GUI-element groundings and step-wise action chains across diverse Android applications. Second, we collect *self-explored trajectories* on AndroidWorld [8] by running an exploration policy on device, producing task rollouts that extend the distribution of observed app states beyond the static AMEX corpus.

Training procedure. Training proceeds in two stages. In Stage 1, we fine-tune the backbone for 1 epoch on explicit chain-of-thought demonstrations, initializing the model with standard text-based reasoning traces and action formatting. In Stage 2, we switch to the latent-CoT objective and train for 3 epochs. The main configuration compresses reasoning into $L = 6$ latent slots and refines them over $K = 3$ APLR passes; component ablations additionally evaluate the 4B model with $L = 9$ latent slots. Unless otherwise specified, the cross-entropy loss weight is $\lambda_{ce} = 0.8$.

C Complete Latent Visualization Diagnostics

This appendix reports the full latent-slot diagnostic views used to support Section 4.4. The analysis is run on a MIRAGE-8B checkpoint evaluated on the AndroidControl IDD split. The diagnostic dump contains 1,012 examples and 3,036 latent vectors after grouping the six latent slots into three adjacent pairs: slots 1–2, slots 3–4, and slots 5–6. This grouping matches the main-paper visualization, where the paired slots form three separated positional clusters.

Figure 6 supports the interpretation that the latent sequence is not a bag of exchangeable hidden states. The three slot groups form separate clusters, especially in UMAP, which means the model uses different parts of the continuous latent sequence differently. In the MIRAGE design, this is desirable: the first slot group can specialize toward observation-level evidence, the middle group toward action rationale, and the final group toward transition prediction. The visualization does not prove this semantic assignment by itself, but it shows that the model has learned a stable positional organization on which such specialization can be built.

Figure 7 shows that action information is present but partially entangled with slot identity. High-frequency tap examples spread broadly, as expected for generic GUI interaction, while more specialized actions such as open-app, swipe, and type appear in more localized regions. The per-slot views are useful because they reveal whether one slot group alone carries the action signal. In our diagnostic run, action separation is visible in each group but remains imperfect, suggesting that action semantics are distributed across the latent sequence rather than isolated in a single slot.

Figure 8 complements Figure 6. Once the slot-specific mean is removed, action structure becomes clearer. Open-app examples form a compact region, swipe examples occupy a separate region, type examples gather in another area, and tap remains a broad central manifold. This behavior is consistent

with the main claim of Section 4.3: MIRAGE latent states encode both where they are in the latent reasoning sequence and what action semantics they support. The slot-position component is strong, but it does not erase action-relevant information; after centering, the residual representation still organizes examples by action type.

D Formal Derivation of Original Coconut

This section formalizes the original Coconut-style latent reasoning procedure [15] that motivates our APLR approximation. Coconut starts from supervised examples with a context c and a visible chain of reasoning split into M ordered chunks $\rho_{1:M}$, followed by a final answer or action a . For mobile agents, c corresponds to the current screenshot, instruction, and interaction history, while a is the serialized GUI action.

Explicit chain-of-thought objective. The ordinary explicit-CoT likelihood factorizes as

$$p_\theta(\rho_{1:M}, a \mid c) = \prod_{i=1}^M p_\theta(\rho_i \mid c, \rho_{<i}) \cdot p_\theta(a \mid c, \rho_{1:M}), \quad (15)$$

where each factor over a chunk denotes the product of token probabilities inside that chunk. Stage 0 of Coconut is equivalent to minimizing the standard next-token loss on the full sequence $(c, \rho_{1:M}, a)$.

Continuous-thought substitution. Coconut introduces boundary tokens $\langle \text{bot} \rangle$ and $\langle \text{eot} \rangle$ for a latent reasoning span. Let $H_\theta(S)$ be the final-layer hidden states produced by the decoder for a mixed sequence S containing both token embeddings and continuous vectors. Let $F_\theta(S)$ denote the hidden state at the last position of S , optionally followed by a projection Π into the input-embedding space:

$$F_\theta(S) = \Pi H_\theta(S)_{|S|}. \quad (16)$$

At curriculum stage m , Coconut replaces the first m visible reasoning chunks by m continuous thoughts $z_{1:m}$. These thoughts are generated recursively:

$$S_0 = [c; \langle \text{bot} \rangle], \quad (17)$$

$$z_i = F_\theta([S_0; z_1; \dots; z_{i-1}]), \quad i = 1, \dots, m. \quad (18)$$

The resulting training sequence is

$$\tilde{S}^{(m)} = [c; \langle \text{bot} \rangle; z_1; \dots; z_m; \langle \text{eot} \rangle; \rho_{m+1}; \dots; \rho_M; a]. \quad (19)$$

Continuous positions are not vocabulary targets. The supervised likelihood at stage m is therefore

$$p_\theta^{(m)}(\rho_{m+1:M}, a \mid c) = \prod_{i=m+1}^M p_\theta(\rho_i \mid c, z_{1:m}, \rho_{m+1:i-1}) \cdot p_\theta(a \mid c, z_{1:m}, \rho_{m+1:M}), \quad (20)$$

with loss

$$\mathcal{L}_{\text{Coconut}}^{(m)} = - \sum_{\ell \in \mathcal{I}^{(m)}} \log p_\theta(\tilde{S}_\ell^{(m)} \mid \tilde{S}_{<\ell}^{(m)}), \quad (21)$$

where $\mathcal{I}^{(m)}$ indexes only discrete supervised tokens after the latent prefix. Increasing m gradually moves supervision from explicit rationales to latent thoughts: $m = 0$ recovers standard CoT training, while $m = M$ leaves only the final answer or action as visible supervision after the latent span.

Serial triangular computation. Equation (18) is inherently serial because z_i must be computed and inserted before z_{i+1} can attend to it. Abstracting away the visible suffix, define a decoder-induced map G_i for the i -th latent update under fixed context c :

$$z_i = G_i(z_1, \dots, z_{i-1}; c), \quad i = 1, \dots, m. \quad (22)$$

Causal attention makes this system strictly triangular: G_i cannot depend on z_i or any future latent z_j with $j > i$. Original Coconut solves Eq. (22) by forward substitution, i.e., it computes z_1 , writes it into the sequence, then computes z_2 , and so on. This exact serial execution is the reference solution that APLR approximates with parallel Jacobi-style refinement.

Algorithm 1 MIRAGE two-stage training pipeline

Input: explicit records (o_t, τ_t, d_t, a_t) and sequential records with optional next frame x_{t+1}

Output: a latent-reasoning GUI policy; discard the Q-Former head at inference time

Stage 1: explicit-thought warmup

- 1: Serialize $[o_t; \langle \text{THOUGHT} \rangle; \tau_t; \langle \text{ACTION_DESC} \rangle; d_t; \langle \text{ACTION} \rangle; a_t]$
- 2: Optimize next-token CE over the structured thought, action description, and action tokens

Stage 2: latent CoT and world-model training

- 3: Replace the $\langle \text{THOUGHT} \rangle$ block with N learned latent slots as in Eq. (4)
 - 4: **for** $r = 0$ **to** $K - 1$ **do**
 - 5: Run one causal VLM pass and update all latent slots synchronously using Eq. (6)
 - 6: **end for**
 - 7: Decode action tokens and compute $\mathcal{L}_{ce}$ by Eq. (8)
 - 8: **if** a valid next frame x_{t+1} exists **then**
 - 9: Extract detached targets $V_{t+1} = \text{sg}(f_{\text{vis}}(x_{t+1}))$
 - 10: Predict $\hat{V}_{t+1}$ from latent states with the Q-Former and compute $\mathcal{L}_{wm}$
 - 11: **else**
 - 12: Mask out the world-model term for this sample
 - 13: **end if**
 - 14: Update parameters with $\mathcal{L} = \lambda \mathcal{L}_{ce} + (1 - \lambda) \mathcal{L}_{wm}$
 - 15: **return** latent-reasoning GUI policy
-

E Appendix Pipeline Pseudocode

Algorithm 1 summarizes the full training pipeline.

F APLR Recovery Proof

We formalize the statement used in Section 3.2.2. The goal is not to prove that a small number of APLR rounds exactly reproduces every serial latent state. Instead, the result is sharper: after K refinement rounds, the first K latent states are exactly identical to the serial rollout, while the remaining latent states have a structured tail error that propagates only through the strictly causal dependency graph.

Setup. Consider one sequence with N latent slots. Each latent state lies in $\mathbb{R}^d$. Let c denote all non-latent context: instruction tokens, history tokens, image embeddings, attention masks, and position encodings. For each latent index i , define G_i as the decoder-induced update that maps the fixed context and all earlier latent states to the hidden state used to replace latent slot i . Causal attention implies

$$G_i = G_i(z_1, \dots, z_{i-1}; c), \quad (23)$$

so G_i cannot depend on z_i or any future latent z_j with $j > i$.

The serial latent rollout is the forward-substitution solution

$$s_i = G_i(s_1, \dots, s_{i-1}; c), \quad i = 1, \dots, N, \quad (24)$$

with the convention that G_1 depends only on c . APLR initializes every latent slot with the same learned latent embedding, written abstractly as $z_i^{(0)} = e_{\text{lat}}$, and performs the parallel Jacobi-style update

$$z_i^{(r+1)} = G_i(z_1^{(r)}, \dots, z_{i-1}^{(r)}; c), \quad i = 1, \dots, N. \quad (25)$$

The word “parallel” means that all right-hand sides in Eq. 25 are evaluated using round- r latent states before any slot is overwritten for round $r + 1$.

Lemma 1 (Causal dependency cone). *After r APLR refinement rounds, latent slot $z_i^{(r)}$ can depend on the initial latent values only through slots with indices at most $i - r$. Equivalently, information from the exact serial prefix can advance by at most one latent position per refinement round.*

Proof. The claim is immediate for $r = 0$ because $z_i^{(0)}$ is itself an initial value. Assume it holds for round r . At round $r + 1$, $z_i^{(r+1)}$ is a function of $z_1^{(r)}, \dots, z_{i-1}^{(r)}$ and the fixed context c . By the induction hypothesis, each predecessor $z_j^{(r)}$ can depend on initial slots only up to index $j - r$. Since $j \leq i - 1$, the largest such index is $(i - 1) - r = i - (r + 1)$. Thus $z_i^{(r+1)}$ can depend on initial values only through slots with indices at most $i - (r + 1)$. This proves the claim by induction. $\square$

Proposition 1 (Exact recovery of early latent states). *For any number of refinement rounds $r \geq 0$, APLR satisfies*

$$z_i^{(r)} = s_i \quad \text{for all } i \leq r. \quad (26)$$

Consequently, after K refinement rounds, APLR exactly recovers the first K latent states of the serial rollout, and after N rounds it recovers the full serial latent sequence.

Proof. We prove the claim by induction on the refinement round r .

Base case. For $r = 0$, the set of indices satisfying $i \leq 0$ is empty, so the claim holds vacuously. No latent has been refined yet, and no equality with the serial rollout is required.

Inductive step. Assume that after round r , $z_j^{(r)} = s_j$ for every $j \leq r$. Consider round $r + 1$ and any latent index $i \leq r + 1$. If $i = 1$, then the causal update has no latent predecessor. Both APLR and serial forward substitution therefore use exactly the same context-only map:

$$z_1^{(r+1)} = G_1(c) = s_1. \quad (27)$$

If $i > 1$, then every predecessor index $j < i$ satisfies $j \leq r$ because $i \leq r + 1$. By the induction hypothesis, all predecessor values used by the APLR update are already equal to their serial values:

$$(z_1^{(r)}, \dots, z_{i-1}^{(r)}) = (s_1, \dots, s_{i-1}). \quad (28)$$

Substituting these equal predecessors into the parallel update gives

$$z_i^{(r+1)} = G_i(z_1^{(r)}, \dots, z_{i-1}^{(r)}; c) = G_i(s_1, \dots, s_{i-1}; c) = s_i, \quad (29)$$

where the final equality follows from the serial recurrence in Eq. 24. Thus the claim holds for round $r + 1$, completing the induction. $\square$

Corollary 1 (Which latent slots remain approximate). *After K refinement rounds, the only latent slots that can differ from the serial rollout are the tail slots*

$$z_{K+1}^{(K)}, z_{K+2}^{(K)}, \dots, z_N^{(K)}. \quad (30)$$

The first K slots have zero serial-approximation error.

Local tail-error expression. The same triangular structure explains the approximation error for the tail slots when $K < N$. Let $\delta_i^{(r)} = z_i^{(r)} - s_i$. Assume each G_i is twice continuously differentiable in a neighborhood of the serial solution. Define the block Jacobian

$$A_{ij} = \begin{cases} \frac{\partial G_i}{\partial z_j} \big|_{s,c}, & j < i, \\ 0, & j \geq i. \end{cases} \quad (31)$$

The matrix A is strictly block-lower-triangular. A Taylor expansion around $(s_1, \dots, s_{i-1})$ gives

$$\delta_i^{(r+1)} = \sum_{j < i} A_{ij} \delta_j^{(r)} + R_i^{(r)}, \quad \|R_i^{(r)}\| \leq C_i \|\delta_{<i}^{(r)}\|^2 \quad (32)$$

for constants C_i determined by local Hessian bounds. Stacking all latent errors yields

$$\delta^{(r+1)} = A \delta^{(r)} + R^{(r)}, \quad \|R^{(r)}\| = O(\|\delta^{(r)}\|^2). \quad (33)$$

Ignoring higher-order terms, the K -round tail error is

$$\delta^{(K)} \approx A^K \delta^{(0)}. \quad (34)$$

Because A is strictly lower triangular, $(A^K)_{ij}$ can be nonzero only when $j \leq i - K$. Thus Eq. 34 has two consequences. First, rows $i \leq K$ are exactly zero, matching the exact-recovery proposition. Second, for a tail latent $i > K$, the remaining error comes only from causal chains of length at least K that connect earlier imperfect initial values to slot i . These are precisely the deep latent states that have not received enough refinement rounds to match the serial rollout.

G How the World-Model Loss Regularizes Tail Errors

The previous section identifies the approximation error left by using $K < N$ APLR rounds: it lives only in the tail latent slots $z_{K+1:N}^{(K)}$. The Q-Former world-model loss does not make these tail slots algebraically identical to the serial rollout. Instead, it adds direct supervision to the components of their error that affect prediction of the next mobile screen in the VLM visual feature space. This is the useful notion of compensation for a mobile agent: the auxiliary loss penalizes tail errors that discard environment-transition information.

Tail-error notation. Let

$$\tau^{(K)} = [\delta_{K+1}^{(K)}, \dots, \delta_N^{(K)}] \quad (35)$$

denote the stacked tail error after K APLR rounds. Let P_ϕ denote the Q-Former aligner, including the row/column query embeddings, cross-attention into latent CoT hidden states, and final visual projection. For a valid next-frame transition, define the detached target feature matrix

$$V^* = \text{sg}(f_{\text{vis}}(x_{t+1})) \in \mathbb{R}^{M \times H_v}. \quad (36)$$

The Q-Former prediction as a function of the latent state is

$$\hat{V}(\tau) = P_\phi(s + \tau; R), \quad (37)$$

where R is the set of valid row/column patch coordinates. For the cosine variant used by default, write normalized predicted and target features as

$$g(\tau) = \text{norm}(\hat{V}(\tau)), \quad u = \text{norm}(V^*), \quad (38)$$

where normalization is applied patchwise. The masked world-model loss is

$$\ell_{\text{wm}}(\tau) = \frac{1}{|\mathcal{M}|} \sum_{j \in \mathcal{M}} (1 - \langle g_j(\tau), u_j \rangle) = \frac{1}{2|\mathcal{M}|} \sum_{j \in \mathcal{M}} \|g_j(\tau) - u_j\|_2^2, \quad (39)$$

because both g_j and u_j have unit norm. Here $\mathcal{M}$ contains only real next-frame patches from samples with a valid next frame and at least one latent slot.

Local expansion. Let J be the Jacobian of the normalized Q-Former prediction $g(\tau)$ with respect to the tail error at $\tau = 0$:

$$g(\tau) = g(0) + J\tau + O(\|\tau\|^2). \quad (40)$$

Substituting Eq. 40 into Eq. 39 yields

$$\ell_{\text{wm}}(\tau) - \ell_{\text{wm}}(0) = \frac{1}{|\mathcal{M}|} \langle J^\top (g(0) - u), \tau \rangle + \frac{1}{2|\mathcal{M}|} \|J\tau\|_2^2 + O(\|\tau\|^3). \quad (41)$$

When the serial latent solution is locally optimized for the world-model target, or when we analyze directions orthogonal to the residual gradient $J^\top (g(0) - u)$, the linear term vanishes. The leading term is then a quadratic penalty on the components of τ that the Q-Former prediction can observe.

Proposition 2 (Q-Former world-model supervision controls future-predictive tail error). *Let U be a subspace of tail-error directions on which the normalized Q-Former Jacobian is locally observable: for all $\xi \in U$,*

$$\|J\xi\|_2 \geq \sigma_U \|\xi\|_2 \quad (42)$$

for some $\sigma_U > 0$. Assume the residual-gradient term in Eq. 41 is zero or treated as a first-order optimization gradient. Then, for sufficiently small tail error,

$$\ell_{\text{wm}}(\tau^{(K)}) - \ell_{\text{wm}}(0) \geq \frac{\sigma_U^2}{2|\mathcal{M}|} \|P_U \tau^{(K)}\|^2 - O(\|\tau^{(K)}\|^3), \quad (43)$$

where P_U projects onto the future-predictive subspace of tail errors that change normalized next-frame feature predictions.

Proof. Using Eq. 41 and dropping the zero residual-gradient term gives

$$\ell_{\text{wm}}(\tau^{(K)}) - \ell_{\text{wm}}(0) \geq \frac{1}{2|\mathcal{M}|} \|J\tau^{(K)}\|_2^2 - O(\|\tau^{(K)}\|^3). \quad (44)$$

Decompose the tail error into a future-predictive component and a locally unobserved component:

$$\tau^{(K)} = P_U \tau^{(K)} + (I - P_U) \tau^{(K)}. \quad (45)$$

By definition of U , the Q-Former prediction is informative on $P_U \tau^{(K)}$, with minimum singular value at least σ_U . Therefore

$$\|J\tau^{(K)}\|_2 \geq \sigma_U \|P_U \tau^{(K)}\|. \quad (46)$$

Substituting this inequality into the previous display yields Eq. 43. $\square$

MSE variant. If the implementation uses the MSE world-model loss instead of cosine distance, the same argument applies without patchwise normalization. Let $\hat{V}(\tau) = \hat{V}(0) + B\tau + O(\|\tau\|^2)$. The masked MSE loss has local expansion

$$\ell_{\text{mse}}(\tau) - \ell_{\text{mse}}(0) = \frac{1}{|\mathcal{M}|} \langle B^\top (\hat{V}(0) - V^*), \tau \rangle + \frac{1}{|\mathcal{M}|} \|B\tau\|_2^2 + O(\|\tau\|^3). \quad (47)$$

Thus any tail-error direction that changes predicted next-frame features is quadratically penalized once the first-order residual term is optimized.

Interpretation. The proposition says exactly which unrefined latent-token errors the Q-Former world-model objective can compensate: errors in the tail slots $z_{K+1:N}^{(K)}$ that are visible to next-frame feature prediction, i.e., errors outside the local nullspace of the Q-Former prediction Jacobian. These directions correspond to latent information that changes the predicted semantic layout of the next screenshot, such as whether a tap opens a menu, whether a typed query changes a text field, or whether navigation moves to a new screen. When the combined loss

$$\mathcal{L} = \lambda_{\text{ce}} \mathcal{L}_{\text{ce}} + \lambda_{\text{wm}} \mathcal{L}_{\text{wm}} \quad (48)$$

is optimized, any such future-predictive tail error increases the objective by at least

$$\frac{\lambda_{\text{wm}} \sigma_U^2}{2|\mathcal{M}|} \|P_U \tau^{(K)}\|^2 \quad (49)$$

up to higher-order terms. Thus the auxiliary world-model loss supplies curvature and gradients for tail latent states that may be weakly constrained by action imitation alone. It does not control tail-error directions that do not affect next-frame visual feature predictions; those directions are either irrelevant to the chosen environment-model target or must be constrained by the action loss and other regularizers.

H System Prompt

Figure 9 shows the complete system prompt provided to the MIRAGE agent at inference time. The prompt defines the role, input format, output format with structured `<THOUGHT>` / `<ACTION_DESC>` / `<ACTION>` tags, and the full command vocabulary. At inference time the `<THOUGHT>` block is replaced by the N latent slots described in Section 3.2.2; the explicit text template here is therefore used only during Stage 1 warmup training.

I Action Space

MIRAGE operates over a compact, typed action space that maps directly to Android system APIs. Each action is a single atomic command; the agent never emits more than one command per step. Table 5 lists all supported action types.

Coordinate convention. All spatial coordinates are normalized to $[0, 999]$ independently in width and height, mapping the top-left corner of the screenshot to $(0, 0)$ and the bottom-right corner to $(999, 999)$. This convention is device-resolution-agnostic: the same action is valid on any screen size and is converted to absolute pixel coordinates at execution time by the environment driver.

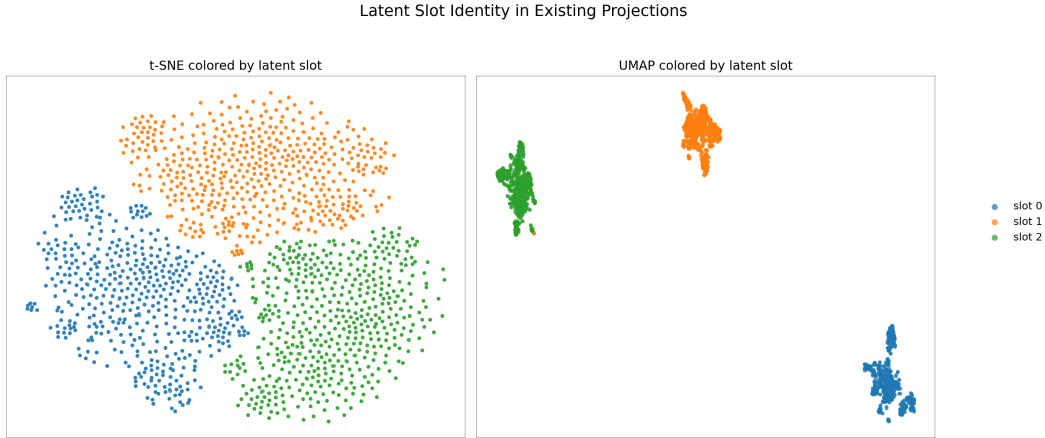


Figure 6: Latent slot identity in existing projections. The left panel colors a t-SNE projection by latent slot group, while the right panel colors a UMAP projection by the same group. Both projections show that adjacent slot pairs form three well-separated regions. This indicates that the latent representation contains a strong slot-position component: early, middle, and late latent slots occupy distinct subspaces before any slot-centering operation is applied.

Table 5: Complete action space of MIRAGE. Coordinates (x, y) are normalized integers in $[0, 999]$ relative to the screenshot dimensions.

Action type	Signature	Description
click	<code>click((x,y))</code>	Tap a visible UI element at screen coordinate (x, y) .
swipe	<code>swipe((x1,y1),(x2,y2))</code>	Swipe from (x_1, y_1) to (x_2, y_2) to scroll or navigate.
drag	<code>drag((x1,y1),(x2,y2))</code>	Drag a slider, handle, or draggable element.
long_press	<code>long_press((x,y))</code>	Long-press to open a context menu or select an item.
type	<code>type("text")</code> <code>type((x1,y1),(x2,y2), "text")</code>	Type into the focused field, or into a specific text box identified by its bounding box.
press	<code>press(KEY)</code>	Press a hardware key (HOME, BACK, ENTER, VOLUME_UP, etc.).
open_app	<code>open_app("AppName")</code>	Launch an installed application by name when that is the intended system action.
wait	<code>wait()</code> / <code>wait(t)</code>	Wait for the UI to load or update; optional timeout t in seconds.
answer	<code>answer("text")</code>	Return a textual answer for a non-GUI information-retrieval question without touching the screen.
call_user	<code>call_user()</code>	Request human takeover when the task cannot be safely continued (e.g. requires biometric authentication).
finish	<code>finish(success)</code> / <code>finish(fail)</code>	Terminate the episode with a success or failure signal.

Action Semantics After Separating Latent Slots

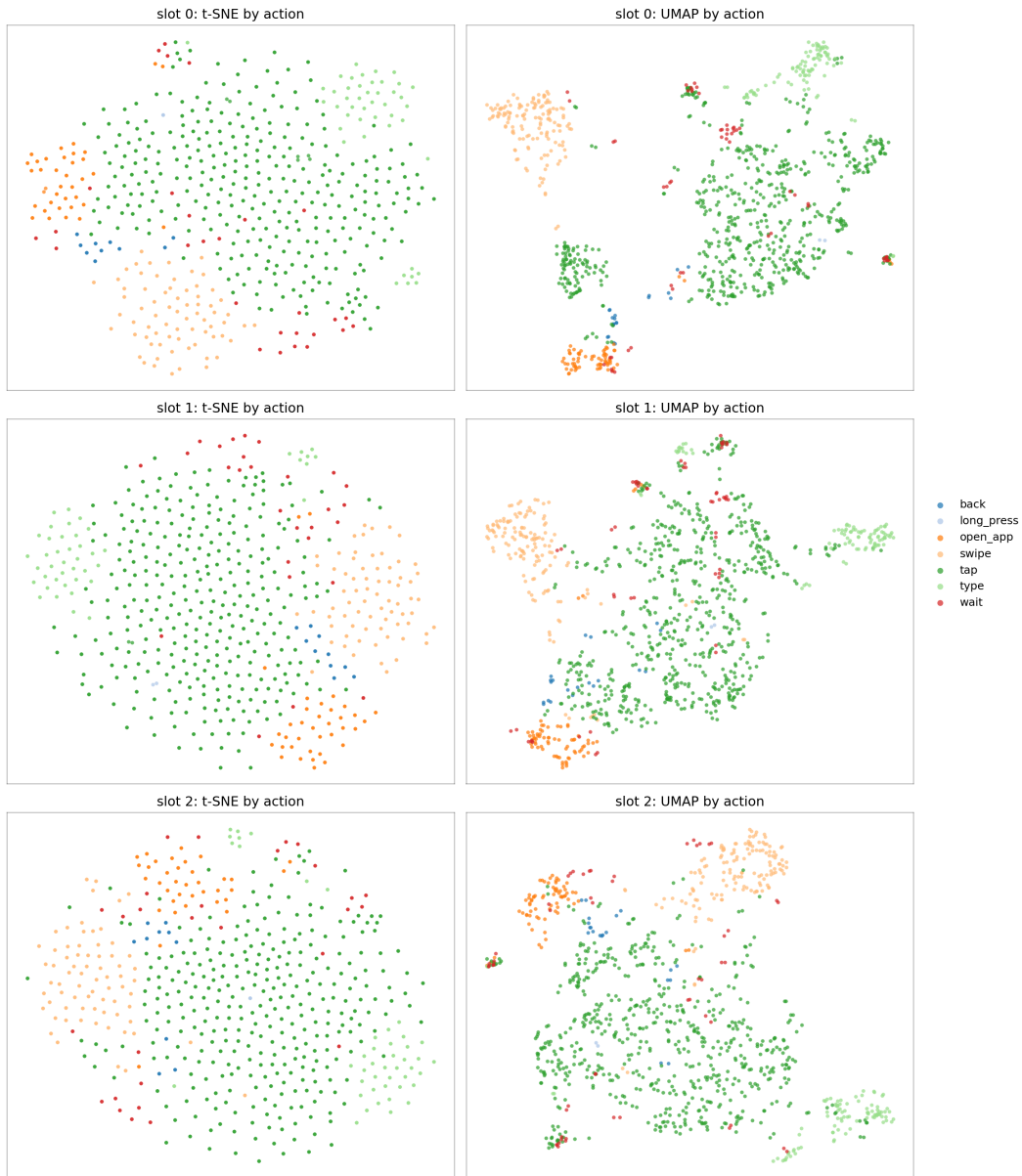


Figure 7: Per-slot action projections. Each subplot projects one slot group and colors points by the ground-truth action type. These plots ask whether action semantics are visible inside each slot group without first removing the slot mean.

Action tokenization. Each action command is serialized as a plain ASCII string (e.g. `click((512,340))`) and appended to the VLM’s output token sequence. The model is trained with next-token cross-entropy over the action tokens $\mathcal{L}_{ce}$ (Eq. 8). No structured JSON, markup, or special delimiters are used; the action string is the only content inside the `<ACTION>` block.

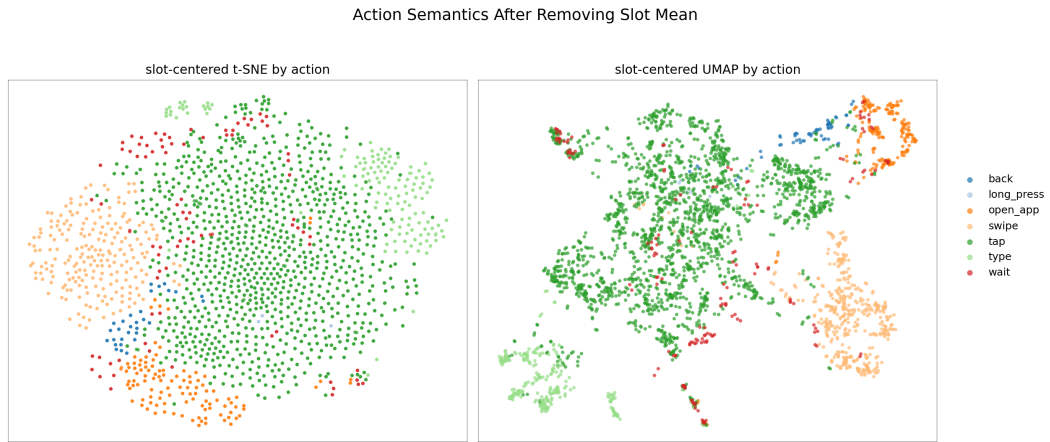


Figure 8: Action semantics after removing the slot mean. The left panel shows slot-centered t-SNE by action type, and the right panel shows slot-centered UMAP by action type. Slot centering subtracts the average representation of each slot group before projection, reducing the dominant slot-identity component.

System Prompt (inference-time template)

You are an Android mobile GUI agent.
 Input: the user task, a summary list of completed past actions, and the current screenshot.
 Choose the single best next action from the current screen only. Do not invent unseen UI.
 Produce one atomic step. Coordinates are normalized integers in [0,999]. Do not emit
 markdown fences or hidden reasoning tags such as <THOUGHT>.

Output:

- Emit only the required blocks using the literal tags '<THOUGHT>', '<ACTION_DESC>', and '<ACTION>'.
- '<THOUGHT>': one English line with exactly

- '1. observation: ... | 2. rationale: ... | 3. predict: ...'.
 - 'observation': describe only the currently visible UI on the current screenshot.
 - 'rationale': combine task goal, prior progress, and the reason this single next action is best.
 - 'predict': describe the immediate visible result right after the action.
- Keep it grounded in the current screenshot and history. Do not mention coordinates. Do not mention prompt structure, annotation metadata, or screenshot-availability phrases such as "next screenshot", "no next screenshot", "one image", or "two images".
- '<ACTION_DESC>': English, one imperative phrase.
 - '<ACTION>': exactly one executable operation command string.

If the task is completed successfully, use 'finish(success)'. If the task should

terminate unsuccessfully, use 'finish(fail)'.

Command format:

- Tap a visible UI element:
 click(<|point_start|>(x,y)<|point_end|>)
- Swipe to move the page or navigate across the screen:
 swipe(<|point_start|>(x1,y1)<|point_end|>, <|point_start|>(x2,y2)<|point_end|>)
- Drag to move a slider, handle, or draggable object:
 drag(<|point_start|>(x1,y1)<|point_end|>, <|point_start|>(x2,y2)<|point_end|>)
- Long press to open a context action or select an item:
 long_press(<|point_start|>(x,y)<|point_end|>)
- Type into the currently focused field:

- type("text")
- Type into a specific text box:
 type(<|box_start|>(x1,y1),(x2,y2)<|box_end|>, "text")
- Press a supported device key:
 press(HOME)
- Open an app directly when that is the intended system action:
 open_app("Settings")
- Wait briefly for the UI to load or update:
 wait() or wait(t)
- Return a textual answer for a non-GUI question:
 answer("text")
- Request human takeover when the task cannot be safely continued:

- call_user()
- End the task:
 finish(success) or finish(fail)

Use exactly one command. Do not wrap it in JSON or natural language. Use English double quotes for string arguments and uppercase English for key names in press(...).

Figure 9: Full system prompt used by MIRAGE. The <THOUGHT> block is a visible text template during Stage 1 warmup; it is replaced by learned latent slots during Stage 2 and at inference time. Coordinates are normalized integers in [0, 999].